From a model bound to the code

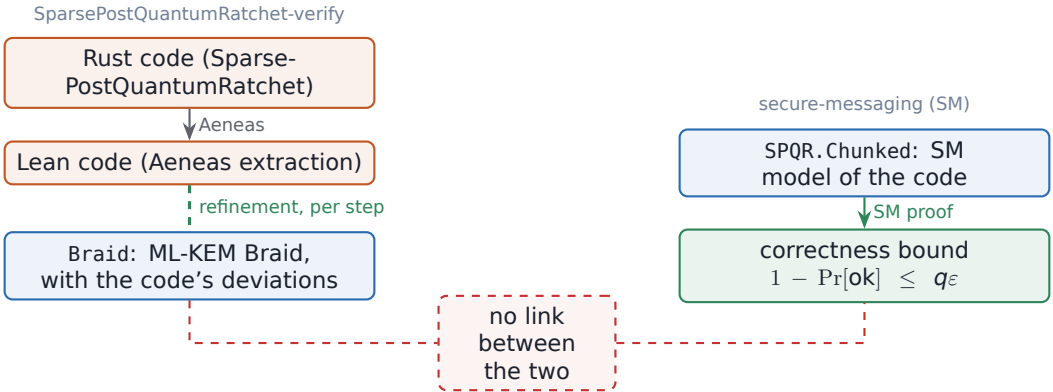
Transferring secure-messaging's SCKA correctness bound to the extracted SPQR code

SpqrBridge

November 2026



Two proofs about SPQR, one gap



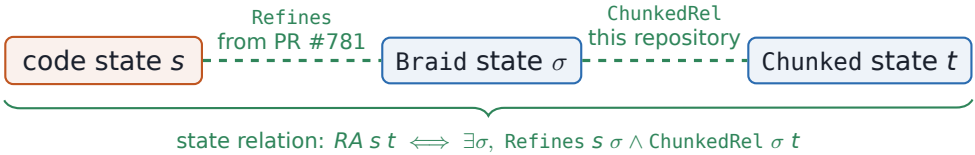
The result

Assume the four facts about the code in `BridgeObligations`, which are not proved yet. Let `adv` make at most q send queries, with $q + 2 < 2^{64}$. Then the probability that the correctness game fails on the code is at most $q\varepsilon$.

```
theorem code_correctness_error_le (hB : BridgeObligations)
    (sampleInitKey : ProbComp InitKey)
    (adv : SCKACorrectnessAdversary Message) (q : ℕ)
    (hq : SendQueryBound adv q) (hq64 : q + 2 < 2 ^ 64) :
    1 - Pr{let x ← correctnessExp (codeScheme sampleInitKey) adv}
          [x = true] ≤
      (q : ℝ≥0∞) * codeKem.correctnessError ProbCompRuntime.probComp
```

- `codeScheme`: the code as an SCKA scheme. Each send draws 64 uniform random bytes, independent of earlier sends, and the code reads at most 64 of them.
- ε: the correctness error of `codeKem`, the KEM of the code's ML-KEM models.
- `BridgeObligations`: four facts about the code, owned by open issues of SparsePostQuantumRatchet-verify (#785, #786, #787). The result is conditional on them.
- No `sorry`. Axioms beyond Lean's three: HKDF, HMAC, one libcrux workaround.

The chain of relations



- *RA*: a code state and an SM state are related through `Braid`. The code state refines some `Braid` state σ (PR #781), and σ is related to the SM state (`ChunkedRel`). Both parties use it.
- `ChunkedRel`: the same of the eleven states, equal epochs and authenticators, byte fields as typed values, equal decoders, and each code encoder's `u32` counter is SM's counter modulo 2^{32}. The header that `Refines` leaves free is taken from the SM state.
- *RM*: `Braid`'s message map `absMsg`, followed by the chunk map $c \mapsto (c.\texttt{index}, c.\texttt{data})$ into SM.
- `send_chunked` and `receive_chunked` cover every arm of both machines.

Where the models differ: the erasure decoders

SM decoder
a set of chunks,
decoded as a set

different
interfaces

Rust decoder
skips points as they arrive,
so its state depends on the order

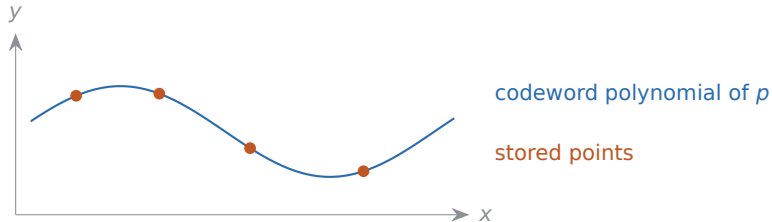
The change, in SM: make the machines generic in a `StreamDecoder` (`empty`, `addChunk`, `decodedPayload`, `Honest`).

The proofs need two laws only:

- the empty decoder is honest for every payload;
- adding a chunk of p to a decoder honest for p keeps it honest, and it then decodes nothing or p (`Lawful`).

SM's set decoder is an instance. The generic theorems take the law as a premise, and the concrete v1 bound keeps its statement.

The Rust decoder is lawful

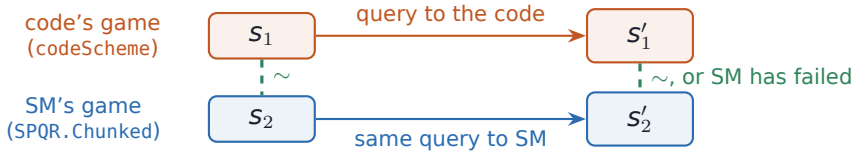


- Honest state: every stored point lies on the slot polynomial of the payload p.
- An honest chunk adds a point of the same polynomial, or is skipped.
- Any K points with distinct x determine the polynomial, so a decoded value is p.
- `polyStreamDecoder_lawful`: one theorem for the four streams, $K \in \{3, 36, 30, 5\}$.
- The order of arrival does not matter for this law.

Moving the bound to the code

- SM's bound comes from one lemma, `correctness_error_le_of_potential`, over a function V_2 on its game states (its "potential").
- The bridge reuses SM's invariant Inv_2 and one-step bounds for the code's game, with a V_1 built from V_2. Here $s_1 \sim s_2$ relates a state of the code's game to a state of SM's game: party states by *RA*, recorded messages by *RM*, and equal key tables, counters and flag.

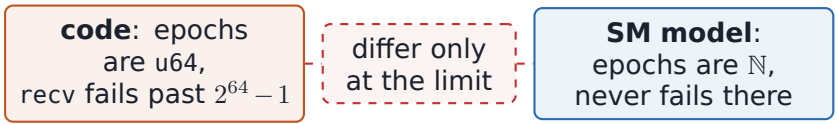
$$V_1(s_1) = \begin{cases} 1 & \text{if } s_1 \text{ has failed} \\ \min\big(1,\ \inf\{\, V_2(s_2) \mid s_1 \sim s_2,\ Inv_2\ s_2 \,\}\big) & \text{otherwise} \end{cases}$$



- After each step, the states are related again, or SM's game failed (slide 9). A failed SM game has $V_2 \geq 1 \geq V_1$, so V_1 inherits SM's bound. The induction counts the remaining sends (`correctness_error_le_of_budgeted_transfer`).

Why 64-bit epochs never overflow

An adversary with fewer than $2^{64} - 2$ sends cannot push an epoch to the overflow point.



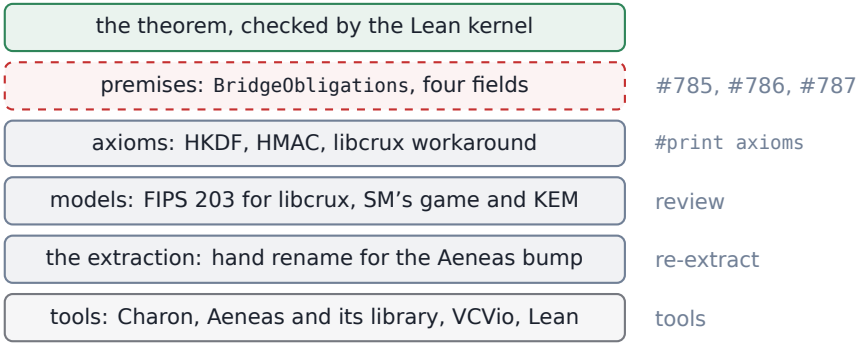
1. At the limit the two machines differ, so they cannot be related there.
2. Every epoch is at most the number of successful sends plus one (`codeGameInv_epochs`).
3. So the step obligations are needed only below a cap $B = 2^{64} - 2$ on the sends: fewer than B for a send, at most B for a receive, since deliveries go on after the last send. With $q + 2 < 2^{64}$, the adversary's q sends stay below B.

The step obligations

init	Every pair of initial states that the code can start in is related to a pair that SM can start in.
send	Both machines read the same 64 random bytes. For every value of those bytes, their results are related.
recv	If SM accepts a recorded message, the code accepts it too, with related results. If SM refuses, nothing is required.

- Two sends read random bytes: key generation (64 bytes) and encapsulation (32 bytes). For these, `codeParams` builds SM's operations from the code's own primitives, so both read the same bytes.
- The other sends read none. The code ignores its bytes, and both results are fixed and related.
- Why nothing is required when SM refuses: SM's game has failed already, and the bound counts that failure.

What the theorem trusts



- ε is not yet ML-KEM's small error. The workaround in `encaps2` only promises a 2080-byte state (#788). The FIPS 203 model of `sampleNTT` stops after 1024 byte triples, but libcrux 0.0.10 does not stop.
- The models and the opaque functions (SHA3, HMAC, libcrux sizes) do not appear in `#print axioms`.
- The bound covers the correctness game: delivery of unmodified recorded messages, with adversarial order and replay. No injected messages, no liveness.

Status and next steps

piece	where	status
code refines `Braid`	spqr-verify, PR #781	proved, one direction
generic stream decoders	SM, `la/generic-decoder`	proved, to propose
`Braid` related to `SPQR.Chunked`	this repository	proved, every arm
Rust decoder is lawful	this repository	proved
transfer of the bound	this repository	proved, for $q + 2 < 2^{64}$
send, recv, tape facts	spqr-verify, #785, #787	open: premises
ε is ML-KEM's error	#788, then a model link	open

spqr-verify: SparsePostQuantumRatchet-verify. SM: secure-messaging.

Next

- Propose the generic stream decoder to secure-messaging.
- Close #785, #786 and #787: each removes a premise.
- #788, so that ε can be identified with ML-KEM-768's error.